

Keeping with your plan, you **do not need to dive into the heavy math of Katz smoothing** (such as the exact Good-Turing discount ratio formulas or the $\alpha(w_{i-1})$ backoff normalization calculations).

Here are the **three main takeaways** about Katz Smoothing that actually matter for your project and your stopping criteria:

1. It is the Canonical Example of a Backoff Model

Unlike Jelinek-Mercer (which is an **interpolated** model that always blends higher- and lower-order probabilities) [87–88], Katz is a strict **backoff** model:

- If the bigram count $c(w_{i-1}w_i) > 0$, Katz uses **only** the discounted bigram probability.
 - If $c(w_{i-1}w_i) = 0$, it **backs off** to use the lower-order unigram probability (scaled by a backoff weight α).
-

2. It Combines Good-Turing Discounting with Backoff

Katz applies the Good-Turing estimate to calculate how much probability mass to subtract from observed counts.

- For small counts $r \leq k$ (where Katz typically sets $k = 5$), counts are discounted according to Good-Turing ratios.
 - For large counts $r > k$, counts are considered reliable and are **not discounted at all** ($d_r = 1$) [59–60].
-

3. Its Empirical Strengths and Weaknesses

- **Strength:** Because Good-Turing accurately estimates the correct average discount for higher counts, Katz performs extremely well on large corpora where high-count n -grams are frequent.
 - **Weakness:** Because it uses strict backoff rather than interpolation, and relies on standard unigram frequencies for $r = 0$, it underperforms on sparse/low-count data compared to Kneser-Ney.
-

Summary Checklist So Far

Algorithm	Type	How it Handles Counts	Main Flaw / Limitation
Jelinek-Mercer	Interpolated [87–88]	Blends models using λ	Linear discounting over-discounts high counts [224–225]
Katz	Backoff	Good-Turing discounting for $r \leq 5$ [57, 59–60]	Pure backoff + raw unigrams underperform on low counts

EXAMPLE REVISITED

Walking through the exact same toy example is a great way to see how **Katz smoothing** works and how **backoff** differs from Jelinek-Mercer's **linear interpolation**.

Setup Recap

- **Vocabulary:** $V = \{\text{"cat"}, \text{"dog"}, \text{"sat"}\}$
- **History:** $w_{i-1} = \text{"cat"}$ (appeared 1 time in training)
- **Seen Target:** "sat" (bigram "cat sat" occurred 1 time)
- **Unseen Target:** "dog" (bigram "cat dog" occurred 0 times)

How Katz Smoothing (Backoff) Handles This Toy Example

Katz smoothing operates in two distinct modes depending on whether an n-gram was seen in the training data.

Step 1: Discount the Observed Counts

Katz subtracts a small amount of probability mass from seen n-grams using a discount factor.

Suppose the algorithm sets the discount ratio for a count of 1 to $d_1 = 0.7$:

- The count for "cat sat" is discounted from 1.0 down to 0.7.

- The subtracted count ($1.0 - 0.7 = 0.3$) becomes the leftover probability mass reserved for all unseen bigrams starting with "cat" .

Step 2: Evaluate the Observed Bigram ("cat sat")

- **Count = 1** (> 0)
- Because "cat sat" was observed, Katz uses **only** the discounted bigram probability:

$$p_{\text{katz}}(\text{"sat"} \mid \text{"cat"}) = \frac{\text{Discounted Count}}{\text{Total History Count}} = \frac{0.7}{1} = \mathbf{0.70}$$

Notice that unlike Jelinek-Mercer, **the unigram probability $p(\text{"sat"})$ is completely ignored** because the bigram count is greater than zero.

Step 3: Evaluate the Unseen Bigram ("cat dog")

- **Count = 0**
- Because "cat dog" was never seen, Katz **backs off** to the lower-order (unigram) model.
- It takes the leftover probability mass (0.3) and distributes it among the unseen words proportional to their unigram probabilities:

$$p_{\text{katz}}(\text{"dog"} \mid \text{"cat"}) = \text{Leftover Mass } (0.3) \times \text{Unigram Share of "dog"} = \mathbf{0.11}$$

Side-by-Side Comparison on the Toy Example

Target Word after "cat"	Count	Jelinek-Mercer (Interpolation)	Katz (Backoff)	Key Difference
"sat" (Seen)	1	$0.6 \times p_{\text{ML}} + 0.4 \times p_{\text{unigram}} = \mathbf{0.78}$	Uses only discounted bigram: 0.70	Interpolation blends both distributions; Backoff ignores unigram for seen n-grams.

"dog" (Unseen)	0	$0.6 \times 0 + 0.4 \times$ $p_{\text{unigram}} = \mathbf{0.11}$	Backs off to unigram using leftover mass: 0.11	Both methods hand down mass to lower-order models for zero counts.
-------------------	---	---	--	--